- Prevents accidental mistakes → Without `@override`, if the method is misspelled, Dart might not detect the error.

c. Comparison: `@override` in Java vs. Flutter
- Both Java and Flutter (Dart) use `@override` to indicate method overriding.
- This helps avoid mistakes when method signatures don't match.
  Java requires explicit method signatures, while Dart is more flexible.

```
class Parent {
    void build() {
        System.out.println("Parent build method");
    }
}

class Child extends Parent {
    @Override
    void build() { // Overriding the parent method
        System.out.println("Child build method");
    }
}
```

- Explanation:
  - `@override` → Ensures `build()` in Child correctly overrides the parent's method.
  - If `build()` was misspelled, Java would throw an error.
- Comparison table :

Feature	**Java**	**Flutter**
Usage of `@override`	Required for overriding methods in OOP	Commonly used in Flutter widgets
Error Prevention	Prevents incorrect method signatures	Helps detect typos in overridden methods
Flexibility	Strict method signatures	More dynamic, but `@override` improves clarity

Step 6 : Exploring the Build Method and BuildContext

a. What is the `build()` Method?
- The `build()` method describes how a widget should appear on the screen.
- It returns a "widget tree", which is a collection of nested widgets forming the UI.
- The `build()` method is called whenever the UI needs to be updated.

```
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      home: Scaffold(
        body: Center(
          child: Text('Hello, Flutter!'),
        ),
```

```
        ),
      );
    }
}
```

- Explanation:
  - `build()` returns a MaterialApp widget, which contains the entire app UI.
  - Inside MaterialApp, there are other widgets like Scaffold, Center, and Text.
  - This structure forms a widget tree, defining how UI elements are arranged.

b. What is BuildContext?
- BuildContext provides information about where a widget is located in the widget tree.
- It allows widgets to access inherited widgets and manage navigation.
- It is an essential parameter in the `build()` method.
- Without BuildContext, widgets wouldn't be able to access inherited properties.

c. How `build()` Keeps the UI Updated
- Whenever Flutter detects changes in the UI, it calls `build()`.
- The `build()` method ensures that the latest UI is displayed.
- Flutter's declarative UI approach means UI is rebuilt instead of manually modified.

d. Comparison with Java: How UI is Built
- In Java (Android), UI updates usually involve modifying View objects.
- In Flutter, UI is declarative and is rebuilt using `build()`.
- Flutter's approach makes UI updates more structured and predictable.

```
@Override
protected View build(Context context) {
    TextView textView = new TextView(context);
    textView.setText("Hello, Android!");
    return textView;
}
```

- Comparison Table :

Feature	**Java (Android)**	**Flutter (Dart)**
UI Structure	Uses XML or Java code for View	Uses build() method to return a widget tree
State Management	Manually updates UI views	UI is rebuilt when state changes
Flexibility	More complex, requires modifying View hierarchy	Simple, UI is updated automatically

Step 7 : What Does Return Do in Flutter

a. What Does return Do in build()?
- Inside the build() method, return specifies what the widget should display.
- It sends the widget back to Flutter for rendering on the screen.
- Flutter relies on return to construct and display UI elements dynamically.

b. Understanding the Returned Widgets

Widget	**Purpose**	**Example**